

◆ BATCH 3.0 ALUMNI ◆

CAREER & GROWTH PLAYBOOK

The complete guide to landing your first AI Engineer role,
standing out in the industry, and building a public presence
that compounds your career.

- | | |
|----|--|
| 01 | Mock Interview Questions |
| 02 | Resume Writing Practice |
| 03 | How to Approach the AI Job Market |
| 04 | How to Stand Out in the Industry |
| 05 | Practical Scenarios |
| 06 | How to Become a Social Influencer for AI |

NISARG KADAM · Lead Agentic AI Architect

4× Global Hackathon Champion · International Keynote Speaker · Singapore

Why this playbook exists

Batch 3.0, you have already done the hardest part. You built. You shipped. You went from RPA developers, freshers, and automation pros to engineers who can architect agentic AI systems end-to-end.

But the bootcamp ending is not the same as the journey ending. In the next 90 days, the difference between the alumni who land roles, get promoted, and build public reputations — and those who don't — will not be technical knowledge. You all already have that.

It will be how well you **communicate, position, network, and build in public**.

This playbook is the field manual. Six sections, designed to be read once cover-to-cover, then referenced weekly as you go through interviews, build your resume, hit the job market, and start showing up online.

HOW TO USE THIS PLAYBOOK

Week 1: Read sections 01 and 02 — prepare interview answers, rewrite your resume

Week 2: Read sections 03 and 04 — map the job market, start standing out

Week 3: Read sections 05 and 06 — practice scenarios, start posting

Week 4 onward: Reference weekly. Iterate. Adjust. Compound.

SECTION ONE

01

Mock Interview Questions

30 questions you will face. Ranked by frequency.
Technical, behavioural, system design, scenario-based.

01 Mock Interview Questions

These are the questions you will hear in 80% of AI Engineer interviews in 2026, grouped into four categories. Below each question is a one-line note on what the interviewer actually wants to hear — not the textbook answer, the answer that signals you have shipped real systems.

Technical fundamentals

Q1. Walk me through what happens when you call an LLM with a prompt. Token by token.

What they actually want to hear: Tokenisation, embeddings, attention, sampling. Bonus points for mentioning context windows and KV cache.

Q2. What is the difference between RAG and fine-tuning? When would you pick one over the other?

What they actually want to hear: RAG for fresh / dynamic data. Fine-tuning for behaviour / style. Don't say one is better — name the trade-off.

Q3. Explain re-ranking in a RAG system. Why is it necessary?

What they actually want to hear: Vector similarity is a lossy first-pass filter. Re-ranking adds query-aware precision. Mention cross-encoders.

Q4. What is the Model Context Protocol (MCP) and why does it exist?

What they actually want to hear: Standardisation problem. Before MCP, every agent integration was bespoke. MCP is to agents what HTTP was to the web.

Q5. How do you evaluate a RAG system's retrieval quality?

What they actually want to hear: MRR, MAP, NDCG for retrieval. RAGAS for end-to-end. Don't just say accuracy.

Q6. Walk me through the LangGraph supervisor pattern.

What they actually want to hear: State graph, supervisor node routes to sub-agents, conditional edges, shared state. Mention real failure modes.

Q7. How do you handle hallucination in production?

What they actually want to hear: Guardrails, structured outputs, grounding, eval harnesses, human-in-loop for high-stakes. Hallucination is a design problem, not a model problem.

Q8. What is prompt injection? How do you defend against it?

What they actually want to hear: Treat all user input as untrusted. Separation of system / user content. Output validation. Mention OWASP LLM Top 10.

System design

Q9. Design a customer support agent that handles 10,000 queries per day across 5 product lines.

What they actually want to hear: Multi-agent supervisor + specialist sub-agents per product. RAG for product docs. Caching layer. Escalation paths to humans.

Q10. Design a RAG system that serves 50 enterprise customers with strict data isolation.

What they actually want to hear: Multi-tenancy via metadata filtering or per-tenant collections. Auth at the retrieval layer. Audit logging.

Q11. How would you A/B test a new prompt in production?

What they actually want to hear: Feature flags, traffic split, evaluation metrics, automated guardrails. Don't ship a prompt change without an eval.

Q12. Your agent's response time has degraded from 2s to 8s overnight. Walk me through your debugging process.

What they actually want to hear: Trace first (LangSmith). Identify the slow node. Check model latency, vector DB latency, tool calls. Don't guess.

Q13. How do you handle cost in an agent that does 100 tool calls per session?

What they actually want to hear: Smaller models for routing. Caching. Prompt compression. Limit recursion depth. Cost is a system constraint, not an afterthought.

Q14. Walk me through how you would deploy a LangGraph agent to production.

What they actually want to hear: Docker, FastAPI wrapper, observability via LangSmith, staged rollout, rollback plan, alerting on key metrics.

Behavioural & situational

Q15. Tell me about an agentic AI project you shipped end-to-end.

What they actually want to hear: Use STAR. Real project from the bootcamp. Quantify: how many users, what was the impact, what would you do differently.

Q16. Describe a time your agent broke in production. How did you handle it?

What they actually want to hear: Show diagnosis, fix, prevention. Vulnerability is a strength here — senior engineers respect honesty about failure.

Q17. Why are you switching from RPA / backend / your current role to AI engineering?

What they actually want to hear: Frame as evolution, not abandonment. Your previous skills compound — they don't compete with AI engineering.

Q18. How do you stay current with AI given how fast the field moves?

What they actually want to hear: Specific sources: papers (Anthropic / OpenAI / Google blogs), specific newsletters, specific GitHub repos you follow. Avoid generic 'I read Twitter'.

Q19. Walk me through how you would explain RAG to a non-technical executive.

What they actually want to hear: Library + librarian analogy. Show you can de-jargonise. Engineers who can't explain to executives stay junior.

Q20. Why should we hire you over a candidate with a CS degree from IIT / NUS?

What they actually want to hear: Lead with shipped work. Pedigree is signal; portfolio is proof. Most companies will pick proof if you give them the option.

Scenario / edge cases

Q21. Your manager wants to add an AI feature to a critical workflow next week. You think it is a bad idea. What do you do?

What they actually want to hear: Surface the risks with data, not opinion. Propose a smaller experiment. Show you can disagree without being a blocker.

Q22. Your RAG system is returning relevant chunks, but the LLM still gives wrong answers. What is happening?

What they actually want to hear: Re-ranking issue, prompt issue, context window crowding, or chunk quality. Methodically isolate.

Q23. Compliance team says you cannot send any user data to OpenAI. Now what?

What they actually want to hear: Self-hosted models, on-prem inference, BYOK with Azure OpenAI, data residency. Show you understand enterprise constraints.

Q24. A junior engineer on your team wrote agent code that works but has no tests. What do you do?

What they actually want to hear: Mentorship moment, not gatekeeping. Show how to write evals together. Build a culture, not just a codebase.

Q25. Your CEO read about agents on LinkedIn and wants 'AI agents in everything by next quarter.' How do you respond?

What they actually want to hear: Don't say no. Say: 'Here is the highest-ROI candidate workflow. Let's prototype that in 2 weeks and evaluate.'

Q26. You have to choose between OpenAI, Anthropic, and an open-source model for a new project. Walk me through your decision.

What they actually want to hear: Use case fit, latency requirements, cost at scale, data sensitivity, fallback / multi-provider strategy. No religious answers.

Q27. Your agent works perfectly in dev but fails 30% of the time in production. What is your first move?

What they actually want to hear: Reproduce the failure with traces. Don't fix what you can't reproduce. Then look at distributional shift between dev and prod inputs.

Q28. How would you convince a sceptical senior engineer that your agentic approach is better than their existing rule-based system?

What they actually want to hear: Don't argue. Build a side-by-side benchmark. Let the numbers speak. Engineers respect data, not pitches.

Q29. You are 6 months into your AI Engineer role and feel impostor syndrome. How would you handle it?

What they actually want to hear: Honest answer: ship something. Impostor syndrome is solved by output, not introspection. Real interviewers respect this answer.

Q30. If you had to teach a non-engineer to build their first agent, what would you do in 30 minutes?

What they actually want to hear: Pick a real problem they have. Use OpenAI Agent Builder UI. Don't open a terminal. Ship something useful in 30 min, not perfect.

HOW TO PRACTICE

- ▶ Pair up with a Batch 3.0 alum. Take turns being interviewer and candidate. 45 mins each.
- ▶ Record yourself answering on Loom or your phone. Watch it back. You will hate it. You will learn.
- ▶ Use ChatGPT / Claude as a mock interviewer for behavioural rounds. Ask it to be tough.
- ▶ For each question above, prepare a 60-second answer and a 3-minute answer. Interviewers control time — you don't.

SECTION TWO

02

Resume Writing Practice

Your resume has 6 seconds to land an interview.
Here is exactly how to make it count for AI Engineer roles.

02 Resume Writing Practice

Most resumes are job descriptions written in third person. That is not a resume — that is a list of responsibilities. A real AI Engineer resume answers one question for the recruiter: **what did you build, and what changed because you built it?**

The 5-second test

Hand your resume to someone who has never met you. Give them 5 seconds. Then take it back. Ask them: what role am I qualified for? If they cannot answer, your resume has failed the only test that matters.

The structure that works in 2026

RESUME STRUCTURE — ONE PAGE, TOP TO BOTTOM

- 1. Header (2 lines):** Name. Role you want, not role you have. Email, LinkedIn, GitHub, location.
- 2. Summary (3 lines max):** Years of experience + domain + 1 marquee outcome. No fluff.
- 3. Selected projects (60% of page):** 3-4 projects. Tech stack + outcome + link to GitHub or live demo.
- 4. Experience (20% of page):** Most recent 2-3 roles. Bullet points starting with verbs. Numbers everywhere.
- 5. Skills (compact strip):** Categorized (LLM frameworks / data / cloud / languages). No 'proficient in MS Word'.
- 6. Education (one line):** Degree, university, year. Skip GPA after 3 years of experience.

Project bullet formula

Every project bullet must follow this formula: **Built [WHAT] using [STACK] that [OUTCOME WITH NUMBER].**

Before vs After examples

WEAK	STRONG
Worked on a chatbot project using LangChain.	Built a multi-tenant customer support agent using LangGraph, ChromaDB, and OpenAI GPT-4o that resolved 73% of L1 tickets autonomously across 1,200 daily queries.
Familiar with RAG and vector databases.	Designed an enterprise RAG pipeline with hybrid search and BGE re-ranking that improved retrieval precision by 41% over baseline cosine similarity.
Helped automate workflows using AI.	Replaced 4 manual UiPath bots with a single LangGraph supervisor agent, reducing process maintenance overhead by 60% while supporting 3× the throughput.

Common resume mistakes

✗ Listing tech you 'know' instead of tech you have shipped with

Recruiters interview the skills section. If you list it, you defend it.

✗ Fluffy summary statements

'Passionate AI enthusiast eager to leverage cutting-edge technologies' tells a recruiter nothing. Replace with: '4 years RPA, 6 months agentic AI. Shipped 3 LangGraph systems. Looking for AI Engineer roles.'

✗ No GitHub link

If you went through Batch 3.0 and don't have a public GitHub with your projects, fix that today. The link itself is a signal.

✗ Generic objective statements

Nobody reads them. Replace with the summary above.

✗ Photo on resume

Singapore: yes. India: optional. US / EU: never. Adjust by market.

✗ PDF named 'Resume_Final_v3_updated.pdf'

Rename to 'YourName_AIEngineer_2026.pdf'. The filename is the first thing the recruiter sees in the email.

✗ No live links to projects

GitHub link, deployed demo URL, or Loom walkthrough video. Make it impossible to ignore.

✗ Listing irrelevant experience

After 5 years of work, drop your college internship. After 8 years, drop your first 2 jobs unless they directly support the narrative.

RESUME PRACTICE EXERCISE

Step 1: Pull your current resume. Highlight every line that does not have a number.

Step 2: For each highlighted line, ask: "What changed because of this work?" Add the metric.

Step 3: Read each line out loud. If it sounds like 4 other people could have written it about themselves, rewrite.

Step 4: Send the new version to 2 Batch 3.0 alumni. Ask them to spot weak claims.

Step 5: Ship it. Iterate after 5 applications, not after every single edit you think of.

SECTION THREE

03

Approaching the AI Job Market

Where the roles are. How to find them. How to apply.
The 4 channels that account for 90% of AI hires.

03 How to approach the AI job market

The AI job market in 2026 is not what you read on LinkedIn. The visible job board listings are 10% of the actual hiring. The other 90% happens through warm intros, GitHub portfolios, public writing, and people who already know you can ship.

If you only apply through job boards, you are competing against 200 other applicants for every role. If you build through the four channels below, you walk into rooms warm, and most of the time the role comes to you.

The four channels that matter

01 Direct outreach to hiring managers

Find AI/ML/Engineering Managers at companies you want to work for. LinkedIn is the playground. Send a 4-line message: who you are, one thing you've shipped, why their company specifically, ask for 15 minutes.

Why it works: Skip the recruiter. Recruiters filter; managers decide. Recruiters don't read your GitHub; managers do.

Action: Send 5 well-researched messages per week. Not 50. Quality of research compounds; volume doesn't.

02 Public proof of work (GitHub + writing)

Your GitHub with 3 polished agentic projects beats 30 generic chatbot tutorials. Pin them. Write detailed READMEs. Add architecture diagrams.

Why it works: If you write one technical blog post per month explaining something you built, after 6 months you will be findable. After 12 months, the right roles will find you.

Action: Aim for 3 polished, public, well-documented projects. Quality over quantity. One repo with 200 well-thought lines beats 10 messy repos with 5,000 lines each.

03 Warm intros from your network

The Batch 3.0 alumni network is your single highest-ROI channel. Twelve students from past batches switched roles — they are now inside companies that may be hiring.

Why it works: When you see someone in your network at a company hiring, ask them for a 20-minute coffee chat. Not a referral. A conversation. Referrals come if the conversation goes well.

Action: Maintain the network. One thoughtful message per quarter to people in your bootcamp keeps the relationship warm. Cold reactivation when you need a job feels obvious and rarely works.

04 Public conversations with the right people

Comment thoughtfully on the LinkedIn posts of senior AI engineers, founders, and hiring managers. Not 'great post' — add a real perspective, ask a sharp question, share data they didn't have.

Why it works: Over 3-6 months, you become familiar to them. When they hire, your name comes up.

Action: 5 thoughtful comments per week on the right 20 people will outperform 100 cold applications over the same period.

Where the AI roles are concentrated in 2026

Don't apply blindly. Different industries hire AI engineers for very different reasons. Understand the segmentation:

Big Tech (Google, Microsoft, Amazon, Meta)

Highest pay. Highest bar. Long interviews. Looking for: research-meets-engineering. If you have a CS degree from a top university or significant open-source contributions, this is reachable. Otherwise, not the highest-ROI starting point.

AI-native startups (under 200 people)

Best for builders. Lower pay than big tech but higher equity. They want people who ship fast. Your bootcamp portfolio matters more here than your degree. Apply through founders, not job boards.

Enterprise transformation (banks, telcos, insurance)

The hidden goldmine. Massive AI budgets, less competition, real problems to solve. They hire RPA-to-AI converts gladly. Singapore, India, UK, Middle East all have major demand.

Consulting (Accenture, Deloitte, EY, KPMG)

Fast learning environment. Lots of variety. Good if you don't yet know what industry you love. Pay is solid; lifestyle is intense. Promotions are well-defined.

Product companies (SaaS, internal AI teams)

The sweet spot for many. Companies like Dell, Salesforce, Shopify, Stripe — not pure AI labs but rapidly adding AI to their products. Real engineering, real users, real impact.

Independent consulting / fractional

Once you have 12+ months of experience and 2-3 case studies, this becomes viable. Singapore's market in particular has strong demand for fractional AI architects at SMEs.

SALARY BENCHMARKS — AI ENGINEER ROLES (2026, MID-LEVEL)

India: ₹ 18-35 LPA (mid), ₹ 35-65 LPA (senior). Bangalore / Hyderabad / Pune lead.

Singapore: S\$ 90k-160k (mid), S\$ 160k-260k (senior). Banks, fintech, product companies.

UK: £ 70k-120k (mid), £ 120k-200k (senior). London concentrated.

USA: US\$ 150k-220k (mid), US\$ 220k-400k+ (senior). Bay Area, NYC, Seattle.

Middle East (Dubai, Riyadh): AED 25-45k/month tax-free for senior roles, growing fast.

Negotiate. Always. The first offer is rarely the best offer.

SECTION FOUR

04

How to Stand Out

What separates the engineers who get ahead from the ones who plateau in their first AI role.

04 How to stand out in the industry

Standing out is not about being the loudest. It is about being the most reliable. The engineer who ships, documents, communicates, and follows through becomes the one everyone wants on their team. Below are seven habits that, compounded over 12 months, separate the people who land lead roles from the ones still applying.

01 Build in public

Every project, every learning, every failure — share it. Not for vanity. For accountability and discoverability. When you show your work over months, people start trusting you before they have ever met you. Hiring becomes a formality.

02 Master one thing deeply before going wide

The temptation in AI is to chase every new release. Resist. Pick one area — RAG, multi-agent orchestration, MCP, evaluation — and become the person your team calls when something breaks. Specialists out-earn generalists until you reach senior staff level.

03 Write. Even badly. Especially badly.

One technical blog post per month, even rough ones, will outperform any course you take. Writing forces you to structure your thinking. Three months in, you will sound clearer in interviews. Twelve months in, recruiters will find you.

04 Become the bridge between business and engineering

Most AI engineers can build. Few can explain. The engineer who can sit in a meeting with the CFO and translate AI capability into business outcomes is the one who gets promoted. Practice this skill ruthlessly.

05 Ship one thing every month, no exceptions

Career momentum is built by output cadence, not output quality on any single piece. A medium-quality project shipped this month beats a perfect project planned for six months from now. Ship monthly.

06 Give before you take

Help people. Answer questions on Stack Overflow. Review pull requests on open-source projects. Send a thoughtful comment to someone you admire. The AI community is small — reputation compounds in both directions, fast.

07 Stay technical even after you become senior

The biggest career trap for engineers who succeed is getting promoted into management and losing technical depth. Even when you lead a team, write code one day a week. Read papers. Stay close to the substrate. The engineers who stay technical at every level become CTOs. The ones who stop building stagnate.

SECTION FIVE

05

Practical Scenarios

Real situations you will face in your first 90 days.
How to handle each one.

05 Practical scenarios

These are scenarios you will actually encounter — not hypotheticals. Each one has cost real engineers real career capital when handled poorly. Here is how to handle each one well.

SCENARIO: Day 1 of your new AI Engineer role

Don't ship code. Listen.

Your first 30 days, your job is to understand the system, the team, and the constraints. Ask 'why does this exist?' for every weird decision you find. Document what you learn. Engineers who try to refactor things in their first week burn political capital they will need later. Earn the right to change things.

SCENARIO: Your manager asks you to add 'AI to our product' with no clear use case

Don't argue. Don't agree blindly. Investigate.

Your first response: 'Let me come back in 5 days with the top 3 candidate workflows where AI would deliver measurable ROI, ranked by feasibility.' Then deliver that document. You have just turned a vague directive into an opportunity to lead a real initiative — and made yourself the person who frames the problem, not just executes.

SCENARIO: A teammate is using ChatGPT to write code that ends up in production

Don't snitch. Educate.

Have a 1:1 conversation. Not 'this is dangerous' — share concrete failure modes. Suggest a code review checkpoint for AI-generated code. Build a practice of 'AI-assisted, human-verified' as a team norm. You become the person who raised the bar without making anyone defensive.

SCENARIO: You are asked to estimate how long an AI feature will take

Multiply your gut number by 2.5.

AI engineering involves more uncertainty than traditional software. Eval design takes longer than you think. Edge cases are 60% of the work. Production hardening is invisible until it's required. Underpromise. Overdeliver. Engineers who consistently miss their estimates lose trust faster than engineers who deliver late but on the date they committed to.

SCENARIO: A senior engineer pushes back on your agentic approach in a design review

Don't argue. Build a benchmark.

'That's a great point. Let me come back in a week with a head-to-head comparison of the agentic approach vs the rule-based system on these three metrics.' Then do exactly that. Engineers who win arguments with data become respected. Engineers who win arguments by talking longer become disliked.

SCENARIO: You discover a bug in an existing AI system that has been live for 6 months

Document. Then talk. Then fix.

Write a one-pager: what the bug is, what data is affected, what the fix looks like, what the risk of the fix is. Take it to your manager. Don't fix it quietly — even if the fix is simple. Quiet fixes that go wrong destroy careers. Documented, communicated fixes build them.

SCENARIO: You're invited to present your work to senior leadership**Lead with the outcome. Not the how.**

Senior leaders don't care about LangGraph supervisor patterns. They care about: what was the problem, what is the impact, what is the next step. 30-second answer to each. Save the technical depth for the appendix slides they will only ask about if curious.

SCENARIO: Your AI feature ships and immediately produces wrong outputs**Be the first to flag it. Always.**

If you find a bug in your own work before anyone else does, you are the engineer who caught it. If someone else finds it first, you are the engineer who shipped it broken. Run your own observability. Watch your own metrics every day for the first two weeks after launch. Beat the customer to the discovery.

SECTION SIX

06

Becoming a Social Influencer

How to build a public presence in AI
that compounds your career and creates opportunities.

06 How to become a social influencer for AI

The word 'influencer' carries baggage — think of it instead as 'building a public reputation.' The AI engineers who do this well don't post for likes. They post because being publicly known for what you build creates opportunities that no resume can.

Inbound job offers. Speaking invitations. Consulting gigs. Communities of peers. Books deals, eventually. All of it starts with the decision to stop consuming content and start producing it.

The 4-step framework

01 Pick one platform and one topic

LinkedIn for professional reach. Twitter / X for technical depth and community. YouTube for long-form authority. Pick one. Master it for 6 months before adding another.

Your topic should be specific enough to be ownable. 'AI' is too broad. 'Agentic AI for enterprise RPA transformation' is ownable. The narrower your initial focus, the faster you become the person people associate with that topic.

02 Post on a fixed cadence, not when inspired

3 posts per week, same days, same times. For 6 months, no exceptions. Inspiration is unreliable. Cadence is the only thing that compounds.

Schedule your posts. Use Buffer or Typefully. Batch-write on Sunday for the week. Treat it like a meeting with yourself — immovable.

03 Write for one person, not for an audience

Pick one specific past version of yourself — say, 'me 12 months ago, when I was an RPA dev about to switch to AI.' Write every post for that person. The advice you wish someone had given you.

Specificity is what makes content shareable. 'Here are 5 AI tips' goes nowhere. 'Here is exactly what I would tell my past self in his last week as a UiPath developer' lands hard.

04 Engage more than you post

Spend 30 minutes per day commenting thoughtfully on other people's posts. Especially people slightly more senior than you. Add real value — disagree carefully, share data, ask sharper questions.

After 90 days of engagement, you will be familiar to several hundred professionals you have never met. When you eventually post your own thoughts, you start with an audience already paying attention.

Content types that work for AI engineers

► The build-in-public log

Share what you are working on this week. The bug you fixed. The architecture decision you made. The trade-off you faced.

Format: 'This week I am building X. Here is the design decision I am wrestling with...'

► The teaching post

Take one technical concept. Explain it like you would to a smart non-engineer. Use analogies. Avoid jargon.

Format: 'I keep getting asked what RAG actually is. Here is the simplest explanation...'

► The contrarian take

Take an industry assumption. Challenge it with data. Be specific. Be defensible.

Format: 'Most people are wrong about prompt engineering. Here is what actually matters...'

► The retrospective

Take a project, a launch, a failure. Tell the story honestly. What worked. What didn't. What you would do differently.

Format: 'We shipped an agent to production last month. Here are the 5 things that broke...'

► The framework / list

Distil your hard-won knowledge into a simple framework. Make it shareable. Make it useful.

Format: 'The 7 questions I ask before adding AI to any workflow...'

► The case study

Pick a real outcome. Show the before, the during, the after. Include numbers.

Format: 'How we replaced 4 RPA bots with one LangGraph agent and reduced maintenance by 60%...'

The 90-day plan

If you start today, in 90 days you will have a small but real presence. Here is the cadence.

Days 1-7 SETUP	Optimise your LinkedIn profile (banner, headline, about section, featured projects). Pick your topic and platform. Write your first 3 posts but don't publish yet.
Days 8-30 POST	Publish 3 posts per week. Don't worry about engagement yet. Just ship. Treat the first month as inventory-building.
Days 31-60 ENGAGE	Continue posting 3 per week. Add 30 minutes daily of thoughtful commenting on other AI engineers' posts. Reply to every comment on your own posts within 1 hour.
Days 61-90 ITERATE	Look at your data. Which posts got the most saves and shares (not just likes)? Double down on those formats. Drop what didn't work. Connect with the people who keep engaging — they are your audience.
Day 90+ COMPOUND	By now, you will have 50+ posts, hundreds of profile visits, and likely a few inbound conversations. Don't slow down. Year 2 is when reputation compounds. Year 3 is when opportunities arrive without you having to ask for them.

A NOTE FROM NISARG

Batch 3.0, you have everything you need. The technical foundation is solid. What comes next is execution — over weeks and months, not days. The engineers from earlier batches who landed roles, switched careers, and built reputations did exactly what is in this playbook. Slowly. Consistently. Without shortcuts.

Read this playbook. Then close it. Then take one action this week. Then another next week.

I am one WhatsApp message away if you need a sanity check, a referral, or just want to talk through a decision. The bootcamp ended; the relationship did not.

Build well.

“Learn to build agents before agents learn to replace you.”

— Nisarg Kadam